RAG-Based Customer Support Assistant

LOW LEVEL DESIGN (LLD)

➢ Objective

To define how the Retrieval-Augmented Generation (RAG) system is implemented internally, including its modules, data flow, workflow, routing logic, and error handling to ensure accurate and context-aware responses using LangGraph, embeddings, and vector databases

➢ Module-Level Design

- **Document Processing Module**
  1. Loads PDF documents from the data source.
  2. Extracts raw text from the document.

- **Chunking Module**
  1. Splits large documents into smaller chunks.
  2. Improves retrieval accuracy and processing efficiency.

- **Embedding Module**
  1. Converts text chunks into vector embeddings.
  2. Uses Sentence Transformer model for semantic representation.

- **Vector Storage Module**
  1. Stores embeddings in ChromaDB vector database.
  2. Enables fast similarity-based search.

- **Retrieval Module**
    1. Performs similarity search on stored embeddings.
    2. Retrieves most relevant document chunks.

- **Query Processing Module**
    1. Accepts user query input.
    2. Converts query into embedding format.

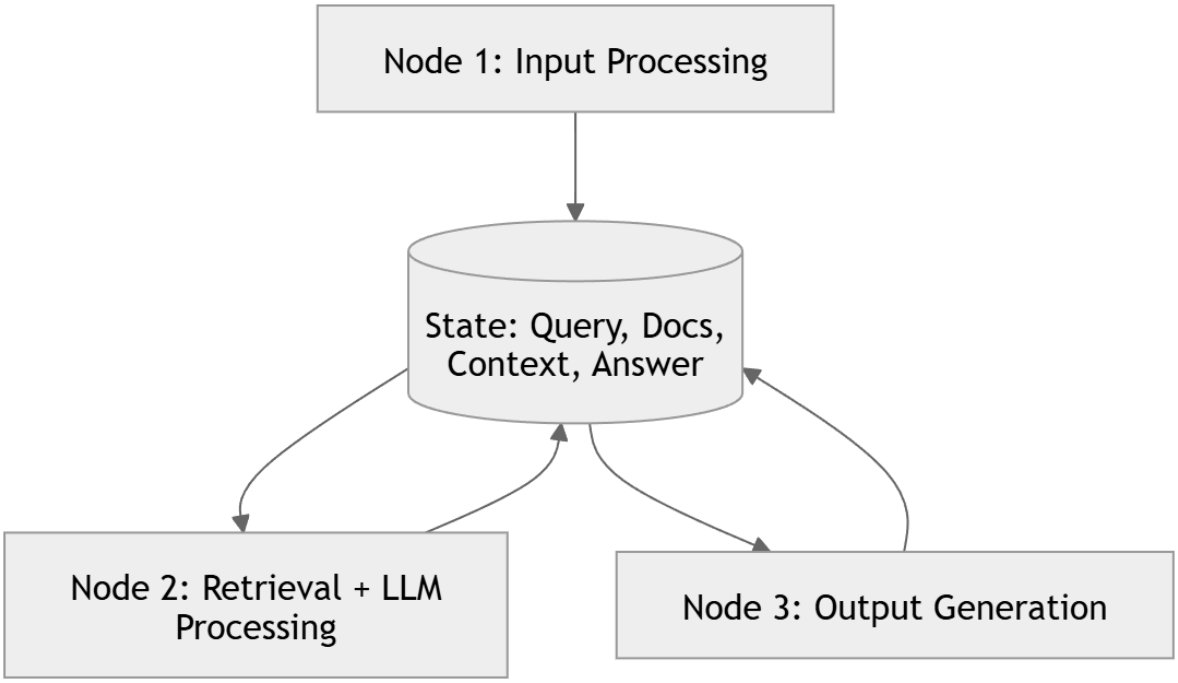
- **Graph Execution Module**
    1. Executes LangGraph workflow.
    2. Manages node transitions and state flow.

- **HITL Module**
    1. Handles Human-in-the-Loop escalation.
    2. Activated when system confidence is low.

➢ Data Structures

- **Document Representation**
  Raw PDF text stored as string format.
- **Chunk Format**
  List of text chunks with optional metadata.
- **Embedding Structure**
  Vector representation (array of floating-point values).
- **Query-Response Schema**
    1. Input: User query (string)
    2. Output: Generated answer (string)
- **State Object (LangGraph)**
    1. Contains query, retrieved documents, and final answer.
    2. Shared across workflow nodes.

➢ Workflow Design (LangGraph)



- **Nodes**
  1. Processing Node → Handles query processing, retrieval, and LLM execution.
  2. Output Node → Returns the final generated response to the user.
- **Edges**
  1. Define the flow between Processing Node and Output Node.
  2. Ensure structured execution of the workflow
- **State**
  1. Central shared object between nodes.
  2. Contains query, retrieved documents, context, and final answer.
  3. Maintains data consistency across the workflow

➢ Conditional Routing Logic

- **Answer Generation Criteria**
  1. Relevant chunks must be retrieved.
  2. LLM generates answer using retrieved context.
- **Escalation Criteria**
  1. Low confidence response
  2. Missing or irrelevant context
  3. Complex or ambiguous query

➢ HITL Design (Human-in-the-Loop)

- **When Escalation is Triggered**
  1. No relevant documents found
  2. Low confidence response from LLM
- **After Escalation**
  1. Query is forwarded to a human operator
  2. Human provides correct response
- **Integration**
  1. Human response is stored in system
  2. Used for future improvement and training

➢ API/Interface design

- **Input Format**
  User Query (string format)
- **Output Format**
  Final Answer (string format)
- **Interaction Flow**
  1. User submits query
  2. System processes through pipeline
  3. Response is returned To user

➢ Error Handling

- **Missing Data**
  Return fallback response message
- **No Relevant Chunks Found**
  Trigger HITL escalation or default response
- **LLM Failure**
  Retry generation or return error message

.